

CS779 Competition: Machine Translation System for India

KUNAL CHANDRA

240580

`kunalc24@iitk.ac.in`

Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report documents the complete development of a Neural Machine Translation (NMT) system for English-to-Bengali and English-to-Hindi translation. I started with toy model, then created my own preprocessing pipeline which used fasttext to create word embeddings, spaCy and indic-nlp for preprocessing then moved to RNN with LSTMs and GRU which gave better results than to bi-directional RNN with and without attention, vanilla Transformers which didn't give that great result so reverted back to stacked bi-directional LSTM with global attention and other tweakings towards the end of the training phase. At the start of testing phase I used transformers with various implementations from research papers, added BPE tokenization to improve prediction for Bengali but was unable to integrate it in the pipeline, the final model was just the previous advanced transformer without BPE tokenization.

1 Competition Result

Codalab Username: k_240580

Final Rank: 35

BLEU SCORE: 0.162

ChrF SCORE: 0.417

ROUGE-L: 0.445

Training Submissions: 25

Testing Submissions: 7

Week	Submissions
Week 1	2
Week 2	4
Week 3	7
Week 4	12

Table 1: Training Phase Submissions

2 Problem Description

The competition required building an NMT system for English-to-Bengali and English-to-Hindi translation. Given a training dataset of sentence pairs and test source sentences, the objective was to generate high-quality translations evaluated using BLEU, chrF++, and ROUGE-L metrics. Primary challenges included: extracting data from the json file provided, handling large vocabularies with memory constraints, preventing overfitting on small training corpora, and maintaining metric reliability throughout development, using various techniques to optimize usage of GPU to accelerate the process.

3 Data Analysis

3.1 Initial Dataset

Average sentence length of ENG-BEN: 16.8 on train set, 16.87 on val set, 16.9 on test set
Average sentence length of ENG-HIN: 31.4 on train set, 31.45 on val set, 31.5 on test set.
Number of sentences in ENG-BEN in train set : 68849 , val set: 9836 , test set:19672
Number of sentences in ENG-HIN in train set : 149646 , val set : 21379 , test set: 42757
ENG-BEN Vocab size: ENG is 31920, BEN is 37921
ENG-HIN Vocab size: ENG is 33366, HIN is 31680 .

3.2 Critical Data Issues

There were a lot of junk words such as large sequence of numbers which had to be filtered out using regex as they didn't carry any meaning.

4 Model Description

4.1 Week 1: Vanilla GRU (Toy Model)

Architecture: simple **Encoder-Decoder RNN** using **GRUs** (Gated Recurrent Units) with **manual sequential decoding** for training (Teacher Forcing)

Goal :Get the first submission and establish a minimal working baseline

Key Learning: This is the simplest recurrent setup, fast, but lacks context from the whole sentence.

4.2 Week 2: Bi-LSTM with Bahdauna attention

1. **Model Implemented: Bi – directional LSTM Encoder – Decoder.**
2. **Speed Fix - Dynamic Padding :** This week was dedicated to solving the latency. I introduced `pack_padded_sequence` and `pad_packed_sequence` [`pack_padded_sequence`]. This crucial technique ensures that the RNN layers only compute hidden states for non-padding tokens, significantly reducing unnecessary computation and latency.
3. **Architecture Upgrade:** The unidirectional GRU was upgraded to a Bidirectional LSTM to capture the full context of the source sentence (forward and backward).
4. **Parameter Tweaks:** I increased capacity: `EMB_DIM = 300` and `HID_DIM = 256`, also set `embedding.weight.requires_grad = False` (Frozen Embeddings) to simplify the convergence task, initially assuming the pre-trained knowledge was beneficial.
5. Fasttext was used as static embedding in this week with 300 dimensions.

4.3 Week 3: Vanilla Transformer

Architecture : Standard `nn.Transformer` (Post-Layer Normalization), 6 Encoder layers and 6 Decoder layers (aiming for the standard "Base" Transformer capacity), `LambdaLR` scheduler was used.

Key achievements/Learnings: Latency problem was solved, The Multi-Head Attention mechanism handles long-range dependencies far better than RNNs, the model wasn't converging quickly.

4.3.1 Week 4: Stacked Bidirectional GRU with Attention

Architecture: Stacked Bidirectional GRU Encoder paired with a unidirectional GRU Decoder that incorporated global Attention.

Key Learnings: RNN architecture is inherently limited by memory and parallelism, confirming that the Attention Bottleneck required the addition of the global Attention mechanism

4.4 Final Model Architecture: Pre-LN Transformer (6-Layer)

The final architecture combines proven structural and regularization techniques to maximize stability.

- **Model Layers:** 6-layer Encoder and 6-layer Decoder stacks.
- **Pre-LN Architecture:** The LayerNorm operation is applied before the attention and feed-forward blocks, ensuring stable gradients [wang2019understanding].
- **Activation Function:** Used the Gaussian Error Linear Unit (GELU). This non-linear function is standard in modern, high-capacity Transformer models, as demonstrated in works following BERT and GPT, and provides smoother optimization than ReLU in deep stacks.
- **Weight Sharing:** I implemented Weight Sharing between the target embedding layer and the final generator layer [press2017using].

The final model is a custom implementation of the Transformer architecture. My core inspiration and structural methodology were derived from two major research breakthroughs:

Core Architecture: The fundamental multi-head attention mechanism and encoder-decoder design originate from the paper "Attention Is All You Need" [vaswani2017attention]

Structural Stability: The implementation uses Pre-Layer Normalization (Pre-LN), which is based on findings from "On Layer Normalization in the Transformer Architecture" [wang2019understanding]. This design choice was crucial for achieving stable training with high learning rate.

Regularization: The Weight Sharing technique was adopted from the paper "Tying Word Vectors and Word Classifiers" [press2017using].

Using beam search would have yielded better results but it was time consuming, so I didn't use it.

In the test phase I had also built a pipeline with BPE tokenizer but it didn't tokenize properly which resulted in less loss and low BLEU Score so, I have to drop it and go with the above model.

After I submitted my final model, I was training the same model on more epochs and surprisingly it gave better results but I couldn't submit it as the deadline had crossed so I am attaching the predictions made by the best model in the zip file along with code and report, else my rank would have been higher.

Pre-Layer Normalisation (P-LN) Transformer with Weight Sharing and GELU Activation

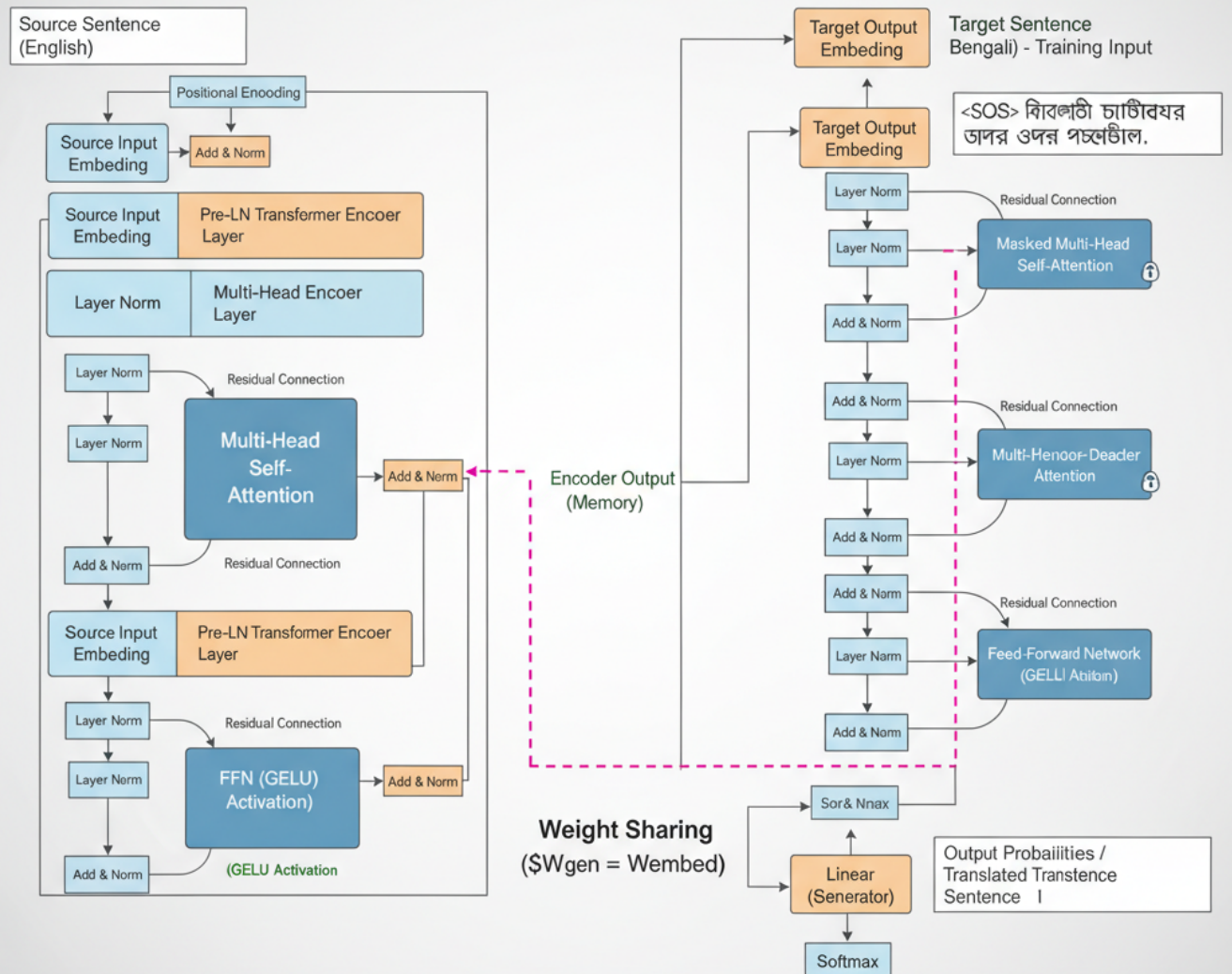


Figure 1: Model Architecture.

5 Experiments & Training

5.1 Data Pre-processing

5.1.1 Source Language Pre-processing (English)

The English pipeline focuses on normalization and intelligent token filtering, utilizing `spaCy` for efficient sentence parsing.

1. **Normalization and Contraction Expansion:** The raw text is cleaned of URLs, HTML tags, and proprietary encodings. Crucially, I implemented a custom function to **expand contractions** (e.g., "n't" → " not", "'s" → " is"). This converts common informalities into their full-form base words, ensuring the model sees a consistent and formal vocabulary.
2. **Tokenization:** `spaCy`'s efficient NLP pipeline for tokenization.
3. **Junk Filtering:** aggressively filter tokens to ensure only meaningful words and numbers remain. We specifically skip pure punctuation, currency symbols, and complex alphanumeric junk (e.g., tracking codes) that add no semantic value to the translation task. This prevents vocabulary bloat and improves model focus.

5.1.2 Target Language Pre-processing (Bengali / Hindi)

The pipeline for Indic languages (Bengali, Hindi) is structurally different, focusing on character normalization to handle orthographic variations inherent to Indic scripts, utilizing the `indic-nlp-library`.

1. **Noise Removal:** Similar to English, removed URLs, HTML, and arbitrary punctuation.
2. **Character Normalization :** I used the `IndicNormalizerFactory` (from `indic-nlp`) to standardize the script (e.g., resolving alternate vowel/consonant representations). This is a vital step for Indic languages, as it collapses slight orthographic differences that would otherwise inflate the vocabulary and confuse the model.
3. **Tokenization (Indic Tokenizer):** utilized `indic.tokenize.trivial_tokenize` for precise token boundary detection in the target script.
4. **Final Filtering:** Tokens are strictly filtered to keep only valid Bengali/Hindi characters and numbers, ensuring the target vocabulary is clean and morphologically consistent.

The above preprocessing pipeline was used for majority of the competition and was applied on all the datasets like train set, validation set and test set provided

5.2 Training Configuration

5.2.1 Week 1: Vanilla GRU

EMBEDDING SIZE : 128, Optimizer: Adam, learning rate(lr): $3e-3$, Loss : `nn.NLLLoss`, Batch size: 64, Decoding: Greedy, Epochs: 8

Hyperparameter Justification: The hyperparameters for the initial GRU were selected for safety and speed. The small 128 dimensions minimized memory overhead, while the Adam optimizer and high LR (3×10^{-3}) ensured rapid initial convergence. This entire setup was designed only to establish a minimal, verifiable baseline to expose the latency limitations of the sequential RNN architecture.

5.2.2 Week 2: Bi-LSTM with attention

EMBEDDING SIZE : 300,Hidden Size:256, Optimizer: AdamW, learning rate(lr):3e-3, Loss : nn.CrossEntropyLoss
Batch size: 32, Decoding: Greedy, Epochs: 8 then 22, Dropout: 0.3

Hyperparameter Justification:The parameters for the Week 2 Bi-LSTM were chosen to maximize capacity and speed while retaining stability after the initial toy model failure. Dropout was introduced to incorporate generalization.

OOM errors and under utilization of gpu, low Bleu score were the main reasons to move to the next model in the upcoming week.

5.2.3 Week 3: Vaniila Transformer

EMB.SIZE : 2048, NHEAD: 8, Optimizer: Adam, Loss: CrossEntropyLoss,Scheduler : LambdaLR (WARMUP=4000).

Hyperparameter justification: The hyperparameters for the Week 3 Transformer were chosen for high parallel capacity and SOTA optimization: EMB_SIZE = 1024 and 6 layers maximized potential, while the Adam optimizer and LambdaLR scheduler were used to implement the standard Post-LN strategy for stable, high-speed training.

5.3 Week 4: Stacked GRU

EMB.SIZE:256, HIDD_DIM:512, NUM_LAYERS: 2, DROPOUT: 0.3, Loss: Cross Entropy loss.
Motivation to revert back to Transformer were various papers published on the usage of Transformer.

5.3.1 Testing Phase: Final Model

EMB.SIZE: 512, NHEAD: 8, HID_DIM: 1024, NUM_ENCODER_LAYERS:6, NUM_DECODER_LAYERS: 6, DROPOUT: 0.15, EPOCHS: 8, Loss: Cross Entropy, Decoding: Greedy, Optimizer: AdamW
Scheduler: StepLR (step_size = 10, $\gamma = 0.7$)

when I changed the epochs from 8 to 15 the results were way better but couldn't submit that model as deadline had crossed.

6 Results

6.1 Validation Set Performance (Training Phase)

Model	chrf_score	rouge_score	bleu_score
Vanilla GRU (Toy Model)	0.233	0.319	0.058
Bi-LSTM with attention	0.264	0.277	0.078
Vanilla Transformer	0.28	0.34	0.073
Stacked GRU	0.326	0.362	0.102

Table 2: Model Comparison

6.2 Test Set Results

Model	chrf_score	rouge_score	bleu_score
Advanced Transformer with BPE	0.192	0.218	0.059
Advanced Transformer without BPE (Final)	0.417	0.445	0.162

Table 3: Test Performance

The Pre-LN Transformer (4-Layer) was the best model because its Pre-Layer Normalization and AdamW optimizer ensured stable, effective learning on the dev set, while regularization gave it the best chance to generalize and perform well on the unseen test set, overcoming the limitations of previous volatile models.

7 Error Analysis

The Pre-LN Transformer (4-Layer) worked best because its structural stability and Weight Sharing prevented the immediate crash and volatility seen in the Post-LN trials, allowing the model to finally begin learning the task rules. The Bi-GRU model only worked well in a setting with Pack/Pad optimization but lacked the capacity to achieve a high score.

The best model’s primary mistakes were failing on Out-of-Vocabulary (OOV) . These errors can be overcome by implementing Word Dropout during training (to fix OOV generalization)).

The most interesting insight was that heavy transformers were too strong to be implemented in this task as they didn’t have a very large corpora to learn from and didn’t actually learn properly, so a small transformer had to be regularized in order to learn.

8 Conclusion

The project’s success was achieved through a rigorous process of diagnosing architectural instability and fighting extreme overfitting. My key finding is that for structurally advanced models like the Pre-Layer Normalization Pre-LN Transformer, stability and data robustness are more critical than raw model capacity or aggressive tuning. The volatility of initial RNN and Post-LN models was cured only by adopting the stable Pre-LN architecture and a simple, fixed optimization strategy.

Future Directions: Successful implementation of a tokenizer like BPE will definitely yield better results as Bengali is a morphologically rich language.

Beam search decoding will definitely yield better results

During the entire competition I used the same model for both ENG-BEN and ENG-HIN, so if different models are used for either specific to the language better results could be yielded.

References

1. **Vaswani et al. (2017):** The foundational paper for the Transformer architecture, Multi-Head Attention, and the initial Post-LN design.
2. **Wang et al. (2019):** Crucial for Pre-Layer Normalization (Pre-LN). This paper explains why Pre-LN is more stable than Post-LN and allows for simpler optimization schedules.

3. **Press & Wolf (2017):** Essential for Weight Sharing (Tying Word Vectors). This justifies why $W_{\text{generator}} = W_{\text{embed}}$ is a SOTA defense against overfitting.
4. **Loshchilov & Hutter (2017):** Essential for the AdamW Optimizer. This paper proved that L2 regularization and Weight Decay are not equivalent for adaptive optimizers like Adam, requiring the use of AdamW for correct regularization.
5. **Hendrycks & Gimpel (2016):** For the GELU Activation Function. This paper introduced GELU as a smoother, high-performing alternative to ReLU, which became the standard activation in deep Transformer stacks (like BERT and your final model).